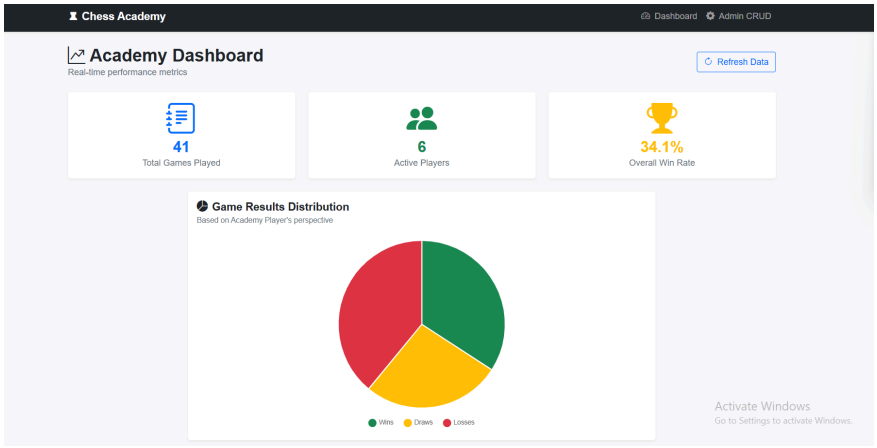
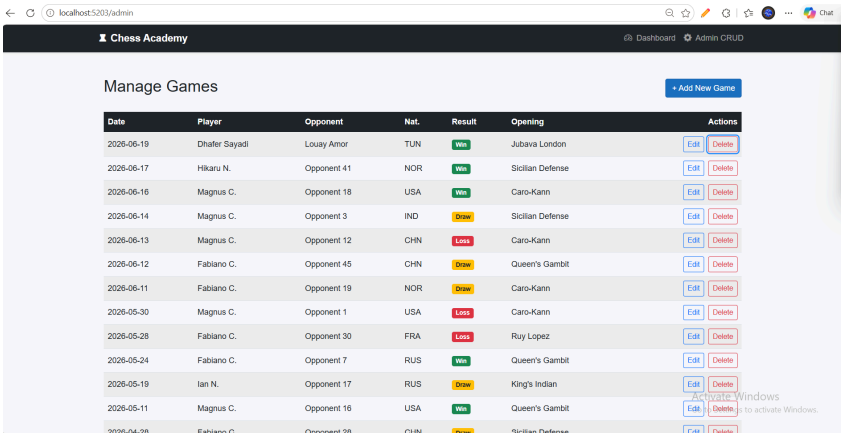
Mohamed Dhafer Sayadi
Mini Projet C#

Chess Academy Tracker

Project Documentation

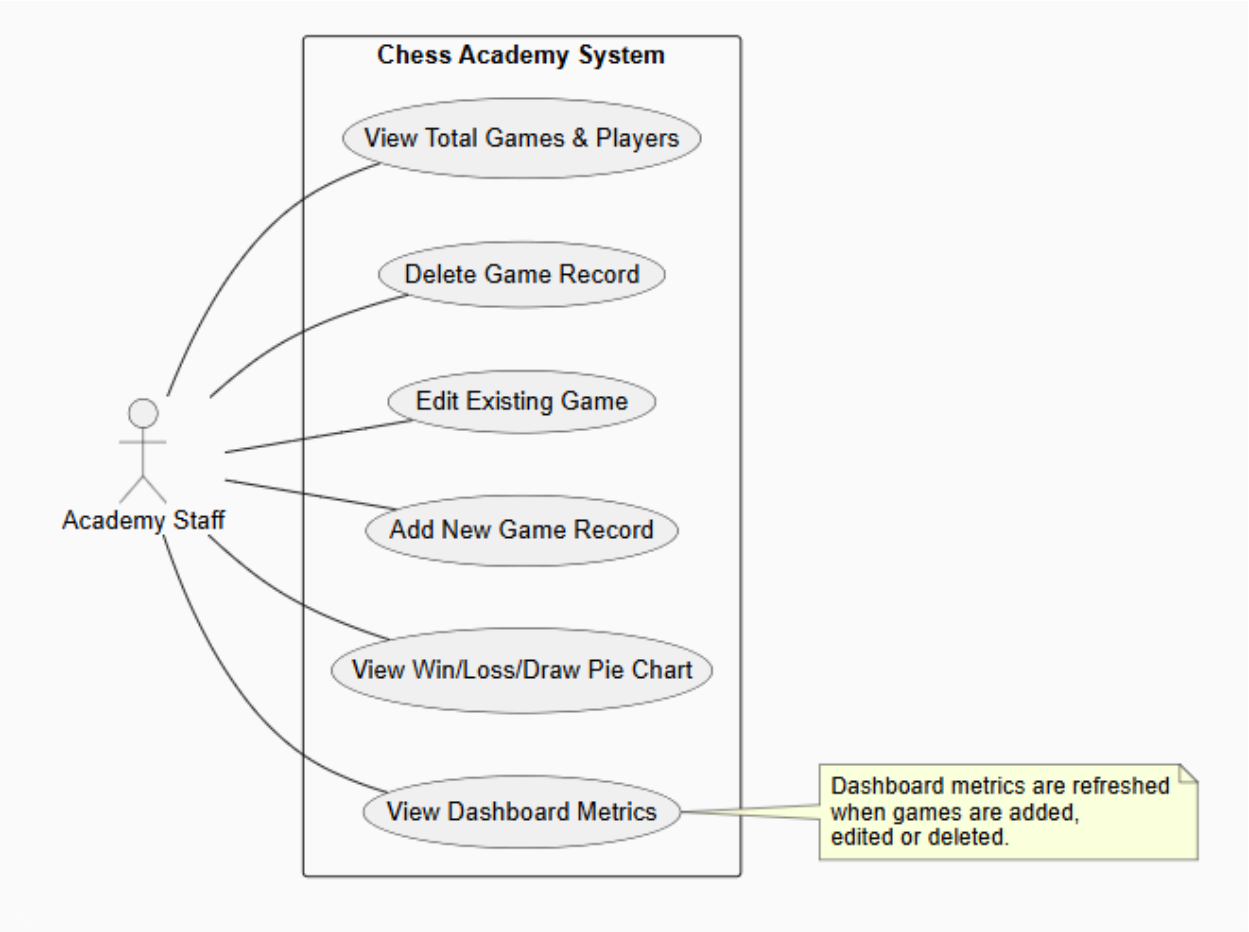




1. Project Overview

The Chess Academy Tracker is a lightweight web application designed to help chess academies monitor player performance and manage game records. It provides real-time analytics through an interactive dashboard and a simple CRUD interface for data management.

2. Use Case Diagram



3. Functional Requirements

- Dashboard Visualization: The system shall display real-time metrics including Total Games, Total Players, and Overall Win Rate percentage.
- Graphical Analytics: The system shall render a pie chart showing the distribution of Wins, Draws, and Losses using Chart.js.
- Create Game Record: The Academy Staff shall be able to add new game records with fields: Player Name, Date, Opponent Name, Opponent Nationality, Result (Win/Draw/Loss), and Opening.
- Update Game Record: The Academy Staff shall be able to edit existing game records to correct or modify data.
- Delete Game Record: The Academy Staff shall be able to remove game records from the system.
- Data Persistence: The system shall store all game records in a SQLite database and maintain data integrity across sessions.
- Dynamic Updates: The dashboard shall automatically reflect changes (new games, edits, deletions) immediately upon navigation from the Admin page.

4. Non-Functional Requirements

- Usability: The interface uses Bootstrap 5 for a clean, responsive design. Navigation is intuitive with a top navbar, and forms use standard HTML inputs for ease of use.
- Maintainability: The codebase follows an ultra-lean architecture with clear separation: Models (data structure), Data (database context), and Pages (UI logic). No complex service layers or repositories were used to keep the code simple and easy to understand.
- Performance: The application uses lightweight libraries (Chart.js via CDN) and a local SQLite database to ensure fast load times. Blazor Server's interactive render mode ensures real-time responsiveness without heavy client-side JavaScript bundles.

5. Technical Stack

- Frontend: Blazor Web App (.NET 8) with Interactive Server rendering
- Database: SQLite with Entity Framework Core
- Visualization: Chart.js (lightweight JavaScript charting library)
- Styling: Bootstrap 5 + Bootstrap Icons
- Architecture: Razor Pages with direct DbContext injection (ultra-lean pattern)